

Capítulo 21

Programação Orientada a Objetos: Polimorfismo

Exercícios de Autorrevisão — Resolução Didática

“Polimorfismo é o coroamento da OO em C++. Significa, literalmente, “muitas formas”: uma só chamada `p->earnings()` executa código diferente dependendo do tipo concreto do objeto apontado por `p`. Este capítulo introduz três novidades inseparavelmente ligadas: funções `virtual`, classes abstratas com funções `virtual = 0`, e o despacho dinâmico via `vtable`. Juntos, eliminam o código `switch` repetitivo que polui programas não-OO.”

1 Exercício 21.1 — Vocabulário do Capítulo 21

Preencha os espaços em cada uma das sentenças (a)–(i).

(a) Tratar um objeto da classe-base como _____ pode causar erros

Resposta: objeto da classe derivada.

A direção oposta da *conversão implícita upcast* (derivada $\rightarrow$ base) é o *downcast* (base $\rightarrow$ derivada), e ele é *perigoso*: pode-se estar lidando com um objeto que não é realmente do tipo derivado, gerando comportamento indefinido. `dynamic_cast` (Cap. 21.8) faz essa conversão com segurança, retornando `nullptr` ou lançando exceção se o objeto não for compatível.

(b) Polimorfismo ajuda a eliminar a lógica de _____

Resposta: `switch`.

Código procedural clássico discriminaria tipos com `switch`: “se for funcionário assalariado, faça A; se for horista, faça B”. Polimorfismo substitui esse `switch` por uma chamada *virtual* única.

```
// ESTILO PROCEDURAL (com switch):
switch (f.tipo) {
    case ASSALARIADO: pag = f.salario * 4; break;
    case HORISTA:     pag = calcularHorista(f); break;
    // ...
}

// ESTILO POLIMORFICO (sem switch):
pag = f.earnings();    // virtual dispatch
```

(c) Se uma classe contém pelo menos uma função virtual pura, ela é _____

Resposta: classe **abstrata**.

`virtual void f() = 0;` declara uma *função virtual pura*. Uma classe com pelo menos uma dessas é abstrata: *não pode ser instanciada diretamente*. Só serve como classe-base para outras classes que implementem todas as funções virtuais puras.

(d) Classes das quais objetos podem ser instanciados são _____

Resposta: classes **concretas**.

Uma classe é concreta se todas as suas funções virtuais puras (herdadas das bases) foram implementadas.

(e) Operador para downcast seguro

Resposta: `dynamic_cast`.

```
Conta *p = obterConta();
ContaPoupanca *cp = dynamic_cast<ContaPoupanca*>(p);
if (cp) {
    // sucesso: p apontava para ContaPoupanca; agora cp aponta para o
    // mesmo
    cp->calculaJuros();
}
// se p NAO apontava para uma ContaPoupanca: cp == nullptr
```

`dynamic_cast` usa a informação RTTI (*Run-Time Type Information*) para checar se o objeto real é do tipo desejado. Para ponteiros, retorna `nullptr` em caso de falha; para referências, lança `std::bad_cast`.

(f) `typeid` retorna referência a _____

Resposta: objeto `type_info`.

```
#include <typeinfo>
Funcionario *f = obterFuncionario();
const std::type_info &t = typeid(*f);
cout << "Tipo real: " << t.name() << endl;
```

`type_info::name()` retorna uma string com o nome do tipo (formato definido pela implementação — `g++` usa nome *mangled*, `c++filt` pode demangle-lo).

(g) _____ envolve usar ponteiro base ou referência para chamar virtuais

Resposta: **Polimorfismo** (polimorfismo de subtipo / dinâmico).

A definição operacional: `Base *p = new Derivada(); p->virt();` chama a versão de *Derivada*, não a de *Base*.

(h) Funções que podem ser sobrepostas usam a palavra-chave _____

Resposta: virtual.

A palavra-chave `virtual` na declaração da base ativa o despacho dinâmico. As derivadas são livres para sobrepor (com ou sem a anotação `override` do C++11, recomendável).

(i) Conversão de ponteiro base → derivada é chamada de _____

Resposta: downcasting.

Up (subir na hierarquia: derivada → base) é sempre seguro e implícito. **Down** (descer: base → derivada) é arriscado e exige `dynamic_cast` ou alternativa equivalente.

Erro Comum de Programação

Um downcast com `static_cast` é sintaticamente válido mas semanticamente perigoso:

```
Conta *p = new ContaCorrente(1000, 2);
// Programador acredita que p eh ContaPoupanca:
ContaPoupanca *cp = static_cast<ContaPoupanca*>(p); // COMPILA!
cp->calculaJuros(); // COMPORTAMENTO INDEFINIDO
```

Use `dynamic_cast` para qualquer downcast em hierarquias polimórficas.

2 Exercício 21.2 — Verdadeiro ou Falso

(a) Todas as virtuais em classe-base abstrata devem ser puras

Resposta: Falso.

Correção: uma classe-base abstrata pode incluir funções `virtual` com implementação, além das puras. O que torna a classe abstrata é ter *pelo menos uma* virtual pura, mas não *todas* precisam sê-lo.

```
class Funcionario {
public:
    virtual double earnings() const = 0; // PURA
    virtual void print() const;         // VIRTUAL COM IMPLEMENTACAO
};
```

Nossa implementação do 21.12 segue exatamente esse padrão: `print` é virtual mas tem corpo na base; `earnings` é = 0.

(b) Referir-se a objeto da derivada com *handle* da base é perigoso

Resposta: Falso.

Correção: o contrário é perigoso: referir-se a um objeto da base com *handle* da derivada. Quando temos `Base *p = new Derivada()`, é seguro — é o cenário normal do polimorfismo. O perigoso seria `Derivada *p = static_cast<Derivada*>(umaBaseDeVerdade)`.

(c) Classe vira abstrata sendo declarada `virtual`

Resposta: Falso.

Correção: *classes* não são declaradas `virtual`. A palavra-chave `virtual` se aplica a *funções-membro* e *base classes* (na herança virtual, Cap. 24). Uma classe se torna abstrata ao conter pelo menos uma *função* virtual pura.

```
virtual class Forma { }; // ERRO de sintaxe

class Forma { // OK: classe abstrata por causa do = 0
public:
    virtual void desenhar() const = 0;
};
```

(d) Se a base declara uma `virtual`, a derivada deve implementar para virar concreta

Resposta: Verdadeiro (com nuance).

A nuance: a regra vale quando a `virtual` da base é *pura*. Se for só `virtual` (sem `= 0`), a derivada *não precisa* sobrescrevê-la: herda a versão da base. Mas pelo enunciado, claramente a questão se refere a virtuais puras.

(e) Programação polimórfica pode eliminar a necessidade de `switch`

Resposta: Verdadeiro.

Esse é o *benefício primário* do polimorfismo, conforme discutiremos em detalhes nos exercícios 21.3 e 21.4: substituir `switch` por chamadas virtuais elimina a maior fonte de manutenção e bugs em sistemas com múltiplos tipos de entidade.

Observação de Engenharia de Software

Não é que `switch` seja sempre ruim: para casos pequenos, com tipos enumerados próximos uns dos outros, `switch` pode ser mais legível que polimorfismo. Mas, quando o número de casos cresce e a lógica específica de cada tipo se torna substancial, polimorfismo ganha duas frentes: organização (cada caso vira a classe correspondente) e extensibilidade (adicionar um caso não requer tocar em código cliente).

Síntese conceitual

A autorrevisão estabeleceu os pilares do polimorfismo em C++:

- **Mecanismo:** funções `virtual` permitem que uma chamada via ponteiro/referência da base execute código da derivada.
- **Abstratos vs concretos:** classes com pelo menos uma `virtual` pura são abstratas (não instanciáveis). Classes que implementam todas as virtuais puras são concretas.
- **Conversões:** upcast é implícito e seguro; downcast requer `dynamic_cast`.
- **RTTI:** `typeid` e `type_info` fornecem informações de tipo em tempo de execução.

- **Benefício primário:** eliminar `switch` sobre tipos — tema dos exercícios 21.3 e 21.4.
